

A static type system for Wavelet, within WIT's limits

Design notes — interview in progress

This note records where the type-system discussion stands. The goal: make Wavelet statically typed, with the core unable to express any type that WIT cannot. Four design questions have been answered so far — **strict** (only WIT types nameable), **no magic builtins**, **full Hindley–Milner inference**, and **arena-only recursion**. One pairing of those answers (strict + no magic builtins) is in deeper tension than it looks; resolving it *is* the type-system design.

The tension: WIT itself cannot type `map`

Here is the fact that forces the issue: **WIT has no generic functions**. `func(list<t>) -> u32` is not legal WIT — every WIT function is monomorphic. So `len`, let alone `map`, has *no* WIT type at all.

That means under a truly literal “only WIT types exist”:

- `map` / `fold` / `len` cannot be **user functions** (their signatures are not WIT types), **and**
- they cannot be **magic builtins** either.

Taken at face value, that leaves them unable to exist — which cannot be the intent. And the chosen **full Hindley–Milner inference** by definition runs on polymorphic type variables ($\forall a...$) internally. So three of the answers only reconcile one way:

Polymorphism is real, but it lives entirely inside the inference engine and the bodies of functions. It is never nameable and never crosses a boundary. A programmer can only *write* WIT types (in `DefType`, `The`, `Fn` params, exports), and every type that reaches a component edge is monomorphic WIT. But `map` is an ordinary, non-magic Wavelet function whose polymorphic type is *inferred*, used only at concrete instantiations, and monomorphized away (à la Rust) before anything is exported.

This honors “strict” as a rule about **what you can name and what crosses boundaries**, and “no magic builtins” by making `map` real code — at the price of admitting the inference engine knows types WIT cannot write down. There is no other consistent reading of the three answers. (The alternative — banning polymorphism outright — also bans `map` as a function, contradicting “no magic builtins.”)

Typeclasses / traits — the deep dive

The problem they solve

Plain HM gives **parametric** polymorphism: a function that works the same for *all* types (`len` ignores what is in the list). But `eq`, `lt`, `to-string`, `add` are *ad-hoc* — they need different code per type. Without typeclasses the only options are (a) magic builtins (rejected), (b) monomorphize and special-case, or (c) pass the operations in by hand. Typeclasses are option (c) made automatic and principled.

The mechanism

A class declares operations over a type variable:

```
Class Eq a { eq: Fn {a a} -> bool }
Instance Eq string { eq: str-eq }
Instance Eq u32 { eq: int-eq }
```

A function that uses `eq` gets an inferred *constrained* type, $\forall a. \text{Eq } a \Rightarrow (a, a) \rightarrow \text{bool}$. The checker resolves the `Eq a =>` constraint at compile time by finding the matching instance, and **elaborates**

it into an extra hidden argument: a **dictionary** — just a record of the instance’s functions — threaded through automatically. Rust traits are the same idea with different ergonomics (coherence rules, associated types, explicit `impl`).

Why this is almost eerily on-brand for Wavelet

Look at what a dictionary *is*: a record of functions. In Wavelet a record of functions is a WIT **record** whose fields are resource-lifted closures — **an ordinary WIT value**. So:

- a **class** is a WIT **interface** (`eq: func(a, a) -> bool`, monomorphized per instance),
- an **instance** is a **component** exporting that interface,
- the **dictionary** passed at runtime is the instantiated component’s exports.

That is the same move already made for macros (“macros are components”) and closures (“closures lift to resources”). Typeclasses do not need to be bolted on — they *fall out of* the Component Model. And crucially for the constraint: every dictionary is a plain WIT value, so the whole mechanism **monomorphizes to WIT at boundaries** just like parametric polymorphism does. Nothing non-WIT escapes.

They are also how the existing magic dies

The design currently hand-waves `eq` as “structural for all data types,” and integer literals “coerce with a range check.” Both are secretly ad-hoc polymorphism. A small fixed class hierarchy makes them honest, non-magic functions:

- `Eq, Ord` → `eq, lt, compare`
- `Show` → `to-string`
- `Num` → `add, sub, mul, ...` and a **defaulting rule** that resolves “integer literal `42` at unknown width” to `s64` — precisely Haskell’s numeric-defaulting story, and exactly the existing `s64`-default + width-coercion behavior, now type-directed instead of ad-hoc.

The costs — clear-eyed

1. **Inference vs. annotations conflict.** Full HM promises near-zero annotations, but typeclasses reintroduce them: ambiguous instances (`to-string(read(s))` — read / show *what* type?) and numeric defaulting force occasional annotations. You cannot have *both* “full HM, no annotations” and rich typeclasses with no friction; the landing spot is “mostly inferred, annotate to disambiguate.”
2. **Coherence in a component world.** Typeclasses normally demand *one* canonical instance per (class, type) globally, or resolution becomes unpredictable. But if instances are components, *who owns the canonical Eq for string*? Two imported components could ship conflicting `Eq string` instances. This is the orphan-instance / coherence problem, and it is *harder* in a composable component ecosystem than in a single Haskell program. This is the real design risk and deserves its own decision.
3. **Higher-kinded classes (Functor / Monad) need kinds ($* \rightarrow *$)** that WIT has zero notion of. They would be purely internal and are the most “exotic” thing that could be added — recommendation: defer or forbid them and stick to single-parameter, kind- $*$ classes (`Eq` / `Ord` / `Show` / `Num`).

Where this leaves the design

A coherent whole emerges: **HM inference with single-parameter typeclasses, polymorphism strictly internal / monomorphized, only WIT types nameable or exported, recursion arena-only.** `eq` / `add` / `to-string` / `map` are all real non-magic functions; nothing non-WIT crosses a boundary. The two open decisions are the scope of the class system and how to handle instance coherence across components.

Open questions (to be answered next)

1. **Resolution.** Comfortable with the only consistent reading — polymorphism is real but internal-only (inferred, monomorphized to WIT at every boundary, never nameable)? Or truly forbid all polymorphism (accepting `map` / `eq` become a closed primitive set)? Or reconsider the tradeoff?
2. **Class scope.** Light fixed set (Eq / Ord / Show / Num, compiler-known)? User-definable classes (instances-are-components, but must solve coherence)? Or no typeclasses (parametric only, `eq` / `to-string` stay structural)?
3. **Coherence.** Decide later? Global coherence (one instance per (class, type) across the composition; conflicts are a compose-time error)? Or local / explicit dictionaries (multiple instances, disambiguate by passing one explicitly)?